Under the Hood: An Introduction to the CH552 Microcontroller

A gentle introduction to computer architecture for ICT students

Version 0.5 · **Date: 14-7-2026** · **Author:** E. vd Oetelaar (edwin@oetelaar.com)

> ***Disclaimer — liability***
>
> *The author accepts no liability whatsoever, in any form, for the use of this document or for any errors it may contain. All information in this document must be verified by the user. This is a work in progress and may contain errors. Use is entirely at your own risk.*

About this document

You have probably already made an LED blink on an Arduino, or read a sensor value in Python. That is a great start — but in both cases a *lot* of machinery was hidden from you. You wrote `digitalWrite(13, HIGH)` and something, somewhere, flipped a physical voltage on a pin. What was that "something"? What actually runs your code?

This document opens the hood. We will use a small, cheap, and refreshingly simple chip — the **CH552** — as our example machine. It is modern enough to be useful (it even speaks USB), but its brain is based on the **8051**, a design from 1980 that is so simple you can hold the whole thing in your head. That makes it perfect for learning.

After this introduction, the logical next step is a small modern RISC-V chip from the same vendor family, for example the **CH32V003F4P6**. It is in roughly the same low-cost class, but architecturally richer: a 32-bit RISC-V core, a different reset flow, a vector table, and more modern peripherals. The intention is to understand the CH552 first, then ask the same questions again on the CH32V003: where does the CPU start, how does C reach `main`, how is the vector table organized, how do we configure minimal GPIO, and how do we then move toward modern peripherals?

The classic Arduino Uno with the AVR ATmega328P is deliberately not the main route. There are already many tutorials for it, and the Arduino layer often hides exactly the architecture details that this learning path tries to expose. Students can explore AVR/Arduino independently as background material.

By the end you should understand, in concrete terms:

- what a microcontroller actually *is*, and how it differs from the computer on your desk;

- what happens between the C code you write and the chip doing something;
- what a **compiler**, **assembler**, and **linker** are;
- the core parts of a CPU: the **program counter**, **accumulator**, **registers**, the **stack** and **stack pointer**, the **status register** and its **flags** (carry, zero, and friends);
- how the CPU makes decisions using **conditional jumps**.

You do not need to memorise the CH552 datasheet. The goal is understanding, not reference.

Table of contents

Part 1 — What is a microcontroller?

A whole computer on one chip

The machine you are reading this on is a *general-purpose computer*. It has a powerful CPU, gigabytes of RAM on separate chips, a disk, a screen, an operating system (Windows, macOS, Linux), and it runs many programs at once.

A **microcontroller** (often abbreviated MCU) takes the *essential* ingredients of a computer — a processor, memory, and a way to talk to the outside world — and squeezes them onto a *single* chip. There is no operating system by default, no hard disk, and usually no screen. It runs exactly *one* program, which you put there, over and over, forever, until the power is cut.

A microcontroller typically contains:

- a **CPU** (the part that executes instructions);

- **program memory** (non-volatile flash memory that holds your program even when powered off);
- **data memory** (volatile RAM that holds variables while running);
- **peripherals** — built-in hardware blocks for talking to the world: GPIO pins, timers, a USB controller, an analog-to-digital converter (ADC), serial ports, and so on.

The whole point is *embedded control*: a microcontroller sits inside a product — a microwave, a keyboard, a toy, a thermostat — and quietly runs it.

Meet the CH552

The **CH552** is an 8-bit microcontroller made by the Chinese company WCH. It is popular with hobbyists and in cheap products because it costs well under a euro, includes a **USB** interface, and can be programmed with free tools.

Key facts about the CH552 (the exact numbers vary slightly across the CH551/CH552 family):

Feature	CH552
CPU core	"E8051" — an enhanced, 8051-compatible core
Data width	8-bit
Clock speed	up to 24 MHz
Program (flash) memory	~16 KB
Internal RAM	256 bytes
On-chip xRAM	1 KB
Peripherals	USB 2.0, GPIO, timers, PWM, ADC, UART, SPI, I²C

"8-bit" means the CPU naturally works with numbers 8 bits wide — one **byte**, values 0–255. When you need bigger numbers, the CPU handles them in several steps, one byte at a time. Compare this to the 64-bit processor in your laptop, which chews through 64 bits at once.

The word **"enhanced"** matters. The original 8051 from 1980 needed 12 clock cycles to execute a typical instruction. The CH552's E8051 core executes about 79% of instructions in a single clock cycle, making it roughly **8 to 15 times faster** than a classic 8051 at the same clock speed. The *architecture* — the registers, instructions, and programming model we are about to learn — is the same. Only the speed changed. This is exactly why the 8051 is a good thing to learn: a 45-year-old design is still shipping in new silicon.

Part 2 — From your code to the chip

Before we look inside the CPU, let's answer the question that Arduino and Python politely hid from you: **how does my code become something the chip runs?**

The CPU only understands numbers

A CPU cannot read C, Python, or English. Deep down it executes **machine code**: a long sequence of bytes stored in program memory. Each instruction is just one or a few bytes. For example, on the 8051 the byte `0x04` means "add one to the accumulator" (the instruction `INC A`). The CPU fetches that byte, recognises it, does the increment, and moves on to the next byte.

Writing programs directly as bytes would be miserable. So we have layers of tools that translate human-friendly text into those bytes.

Assembly language and the assembler

The thinnest possible layer above raw bytes is **assembly language**. It gives each machine instruction a short readable name (a *mnemonic*). Instead of `0x04` you write `INC A`. Assembly is a near one-to-one, human-readable form of machine code.

The tool that turns assembly text into machine-code bytes is called an **assembler**. Assembly → machine code. That is (almost) all it does.

```
; A tiny piece of 8051 assembly
    MOV  A, #5      ; put the number 5 into the accumulator
    INC  A          ; A is now 6
    ADD  A, #10     ; A is now 16
```

Each of those lines corresponds directly to a handful of bytes in memory. The assembler's job is essentially a lookup: turn each mnemonic into its byte(s).

High-level languages and the compiler

Assembly is readable, but still painfully low-level — you manage every register and memory address by hand. That is why we usually write in a **high-level language** like C. C lets you write `x = x + 10;` without caring which register holds `x`.

A **compiler** is the tool that translates a high-level language into a lower-level one — typically into assembly (which is then assembled) or straight into machine code. For the CH552, the standard free compiler is **SDCC** (the Small Device C Compiler), which knows how to target the 8051 family.

So the chain so far is:

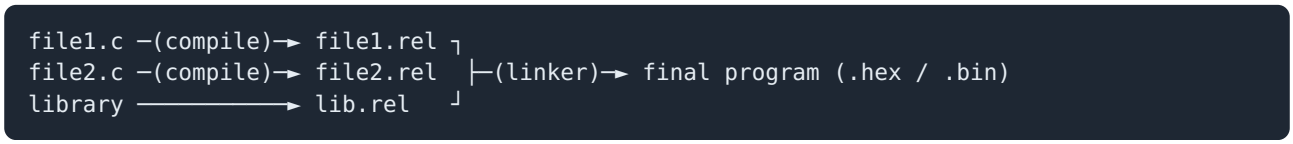
```
C source  ──(compiler)──►  assembly  ──(assembler)──►  machine code
```

A compiler does far more than a simple lookup. It has to decide which variables live in which registers, translate `if` statements and loops into jumps, and optimise. One line of C can become a dozen machine instructions.

The linker

Real programs are split across several files, and they use pre-written building blocks (libraries) — for example, code that knows how to talk to the USB peripheral. Each file is compiled separately into an *object file* (a chunk of machine code with some blanks, because file A doesn't yet know where file B's functions ended up in memory).

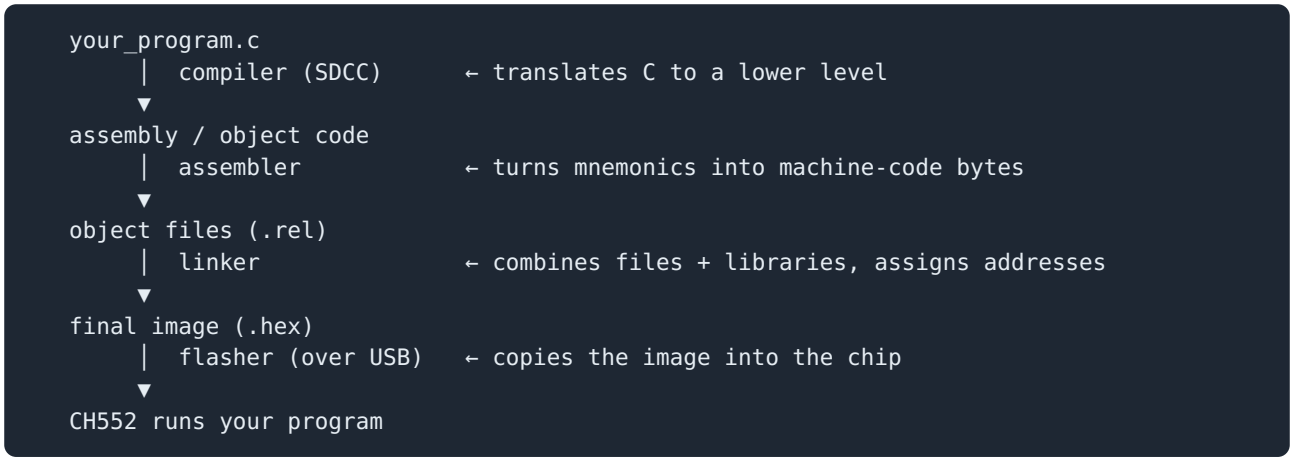
The **linker** is the tool that stitches all the object files and libraries together into one final program, filling in those blanks — deciding the final memory address of every function and variable so the pieces can call each other. Its output is a single image ready to be placed in the chip's flash memory.



Flashing

That final program still lives on your PC. The last step, **flashing** (or *programming*), copies it into the microcontroller's flash memory over USB. From that moment the CH552 will run *your* program every time it powers on. The CH552 conveniently has a built-in USB bootloader, so no special hardware programmer is needed.

The whole pipeline



When you clicked "Upload" in the Arduino IDE, *all* of this happened in the background. Now you know what those layers were.

Part 3 — Inside the CPU: the programming model

Now we open the CPU itself. A CPU is, at heart, a very fast, very obedient, and very stupid machine that repeats one loop forever:

> ***fetch** the next instruction → **decode** what it means → **execute** it → repeat.*

This is called the **fetch–decode–execute cycle**, and it never stops while the chip is powered.

To do its work the CPU has a handful of tiny, very fast storage locations built into the silicon called **registers**. Registers are *not* RAM — there are only a few of them, they have fixed jobs, and they are the fastest storage the CPU has. The collection of registers and how they behave is called the CPU's **programming model**. Let's meet the 8051's, which is the CH552's.

The Program Counter (PC)

The **program counter** is a register that holds the memory address of the *next* instruction to fetch. Think of it as the CPU's finger pointing at a line in the program.

The fetch–decode–execute loop works like this: the CPU reads the instruction the PC points at, and *automatically advances* the PC to the following instruction. So by default the CPU marches straight through your program, one instruction after another, in order.

The PC is what makes a **jump** possible: if we forcibly *change* the value in the PC, the CPU will continue from a different place. That single idea — overwriting the PC — is the basis of every loop, `if`, and function call in every program ever written. We return to it in Part 5.

On the 8051 the PC is 16 bits wide, which is why the chip can address up to 64 KB of program memory (2^{16} = 65 536 addresses).

The Accumulator (A)

The **accumulator**, written **A** (or **ACC**), is the single most important register on the 8051. It is the CPU's "scratchpad" — the default place where arithmetic and logic happen.

On many older CPUs, including the 8051, most calculations *flow through* the accumulator: you load a value into A, do something to it, and the result appears in A. For example, to add two numbers you put one in A and add the other *to* A; the sum lands back in A.

```
MOV  A, #20     ; A = 20
ADD  A, #22     ; A = A + 22  → A = 42
```

The name is historical: it "accumulates" running results. A modern CPU has many equal general-purpose registers, but the 8051 leans heavily on this one special register, which actually makes it *easier* to learn — there is one obvious place where the action happens.

There is also a helper register **B**, used mainly together with A for multiplication and division.

The general-purpose registers R0–R7

Besides A and B, the 8051 gives you eight general-purpose registers named **R0 through R7** for holding working values. Cleverly, these are not separate hardware — they are simply the first bytes of internal RAM, exposed under handy names.

In fact the 8051 has **four banks** of R0–R7, and you can switch which bank is "active" using two bits in the status register (explained below). This is a neat trick for quickly swapping a whole set of working variables — for instance when an interrupt occurs, so the interrupt routine doesn't clobber the main program's registers.

```
MOV  R0, #100   ; store 100 in register R0
MOV  A, R0      ; copy R0 into the accumulator
```

Memory: a quick map

It helps to know where things live. The 8051 keeps program and data in *separate* spaces (this is called a **Harvard architecture**, in contrast to the **von Neumann** design of your PC where code and data share one memory):

- **Program memory (flash):** your instructions. Read-only at run time. ~16 KB on the CH552.
- **Internal RAM:** 256 bytes of fast on-chip data memory. The bottom 128 bytes hold the register banks, a bit-addressable area, and the stack; the top 128 bytes overlap in address with the special-function registers (below) but are reached differently.
- **Special Function Registers (SFRs):** a block of registers at addresses 80h–FFh that are the *control panel* of the chip. Writing to an SFR does something physical — sets a pin high, starts a timer, configures the USB block. A, B, the stack pointer, and the status register all live here. On the CH552, WCH added extra SFRs to control its custom peripherals (like USB) on top of the standard 8051 set.
- **xRAM:** 1 KB of extra on-chip RAM for larger data.

The important takeaway for now: **registers are few and fast; RAM is larger and slightly slower; flash holds the program.**

Part 4 — The status register and its flags

When the CPU does a calculation, the *result* goes into the accumulator — but the CPU also records some *facts about* that result in a special register. On the 8051 this is the **Program Status Word (PSW)**, the chip's **status register**.

The PSW is a single byte, and each bit is a separate **flag** — a yes/no signal about the most recent operation or the CPU's current mode. These flags are how the CPU "remembers"

things like *did that addition overflow?* and, crucially, they are what conditional jumps look at to make decisions.

Here is the 8051 PSW, bit by bit:

Bit	Name	Meaning
PSW.7	**CY**	**Carry flag**
PSW.6	**AC**	Auxiliary carry (carry out of the low nibble; used for BCD arithmetic)
PSW.5	**F0**	General-purpose user flag (free for you to use)
PSW.4	**RS1**	Register-bank select bit 1
PSW.3	**RS0**	Register-bank select bit 0
PSW.2	**OV**	**Overflow flag**
PSW.1	—	User-definable flag
PSW.0	**P**	**Parity** of the accumulator (1 if A has an odd number of 1-bits)

Let's look at the ones that matter most for beginners.

The carry flag (CY)

The **carry flag** captures the "extra" bit that falls off the top when an 8-bit calculation doesn't fit in 8 bits.

Remember A holds a single byte: values 0–255. What happens if you add 200 + 100? The true answer is 300, which does not fit in 8 bits. The accumulator keeps the low 8 bits (300 − 256 = 44) and the **carry flag is set to 1** to signal "there was an overflow past 255; a 1 carried out of the top."

```
    MOV  A, #200
    ADD  A, #100    ; true result 300; A = 44, CY = 1  (carry out!)

    MOV  A, #10
    ADD  A, #20     ; result 30 fits fine; A = 30, CY = 0
```

This is exactly like carrying the 1 when you add 7 + 5 = 12 by hand: you write 2 and carry 1. The carry flag is how the CPU chains byte-sized additions together to handle bigger numbers, and how it reports unsigned overflow.

The zero flag — a special case worth understanding

Here is a subtlety that surprises people, and it is a great teaching moment. On many CPUs there is a dedicated **zero flag** that is automatically set whenever a result comes out equal to zero. The classic 8051 **does not have a zero-flag bit in its PSW.**

Instead, the 8051 tests the accumulator for zero *directly*, at the moment of the jump, with the instructions `JZ` ("jump if A is zero") and `JNZ` ("jump if A is not zero"). The concept of

"is the result zero?" absolutely exists — it is simply evaluated on demand rather than stored in a flag bit.

So when we talk about "the zero flag" as a general architecture concept, keep in mind it is a *concept* that different CPUs implement differently. On the 8051 the question "is it zero?" is answered by the `JZ` / `JNZ` instructions looking straight at the accumulator. This is a good reminder that "computer architecture" is a family of related designs, not one fixed rulebook.

The overflow flag (OV) and parity (P)

The **overflow flag (OV)** is the *signed* cousin of the carry flag. When you treat bytes as signed numbers (−128 to +127), OV is set if the result doesn't fit in that signed range. Carry is for unsigned overflow; overflow is for signed. Beginners can safely file this away until they study signed arithmetic in detail.

The **parity flag (P)** simply reports whether the accumulator currently contains an odd or even number of 1-bits. It was historically used for cheap error checking in communication. You will rarely touch it directly.

The bank-select bits (RS1, RS0)

Remember the four banks of R0–R7? **RS1 and RS0** are the two PSW bits that choose which bank is active. `00` selects bank 0, `01` bank 1, and so on. These bits are part of the "status" because they are part of the CPU's current *mode*, not a fact about a calculation.

Part 5 — Making decisions: conditional jumps

We now have all the pieces to explain how a CPU makes *decisions* — how a dumb straight-line executor produces `if` statements, loops, and branches.

Unconditional jumps

Recall from Part 3 that the program counter normally advances by itself, so instructions run in order. A **jump** instruction overrides that: it loads a new address into the PC, so execution continues somewhere else. An **unconditional jump** always jumps:

```
        SJMP  somewhere    ; always jump to the label 'somewhere'
```

This alone gives you an infinite loop — the heart of almost every microcontroller program, which typically runs forever:

```
loop:   ; ... do something ...
        SJMP  loop         ; go back to the top, forever
```

Conditional jumps

A **conditional jump** jumps *only if some condition is true*, and otherwise falls through to the next instruction. The condition is usually a **flag** in the status register, or a direct test of the accumulator. This is the single mechanism behind every `if`, `while`, and `for` in your C code.

The common 8051 conditional jumps:

Instruction	Jumps if…
`JZ`	the accumulator is zero
`JNZ`	the accumulator is not zero
`JC`	the **carry** flag is 1
`JNC`	the carry flag is 0
`JB`	a specified bit is 1
`JNB`	a specified bit is 0
`CJNE`	two values are not equal (and it also sets carry to show which was larger)
`DJNZ`	after decrementing a register, it is not yet zero

From C to jumps: a worked example

Consider this ordinary C code:

```
if (x == 0) {
    y = 1;
}
```

The CPU has no `if` instruction. The compiler turns this into a *test* followed by a *conditional jump*. In 8051 assembly it might look like:

```
        MOV  A, x          ; load x into the accumulator
        JNZ  skip          ; if A is NOT zero, jump past the body
        MOV  y, #1         ; (runs only when x == 0)
skip:   ; ... program continues here ...
```

Read it carefully. To run the body "when x equals zero," the compiler jumps *away* when x is *not* zero. That inversion is completely normal in machine code and is exactly the kind of bookkeeping a compiler does for you.

Loops with a counter

`DJNZ` ("decrement and jump if not zero") builds a counting loop in a single instruction — it is a favourite on the 8051:

```
        MOV  R2, #10       ; repeat 10 times
again:  ; ... loop body ...
        DJNZ R2, again     ; R2 = R2 - 1; if R2 ≠ 0, jump back to 'again'
        ; falls through to here after 10 iterations
```

This is the assembly-level skeleton of a `for (i = 10; i != 0; i--)` loop. Every loop you have ever written compiles down to a conditional jump like this.

Part 6 — The stack and the stack pointer

The last core concept is the **stack** — the mechanism that lets one piece of code call another and safely return.

What problem does the stack solve?

Suppose your `main` code calls a function `readSensor()`. When `readSensor` finishes, the CPU must return to *exactly* the instruction after the call in `main`. But the CPU only has one program counter. How does it remember where to come back to? And what if `readSensor` itself calls another function? We need to remember a *chain* of return addresses, most-recent-first.

The answer is a **stack**: a region of memory used in **last-in, first-out (LIFO)** order, exactly like a stack of plates. You **push** something on top; you **pop** the top one off. The last thing pushed is the first thing popped.

The stack pointer (SP)

Yes — the 8051, and therefore the CH552, has a **stack pointer**. It is an 8-bit SFR called **SP** that holds the internal-RAM address of the current top of the stack.

- **Push:** the CPU increments SP, then writes a byte at that address. (On the 8051 the stack grows *upward*, toward higher addresses.)
- **Pop:** the CPU reads the byte at SP, then decrements SP.

On reset, SP starts at address `07h`, so the stack begins just above the register banks in internal RAM.

`CALL` and `RET`: the stack in action

When the CPU executes a `CALL` instruction (a function call), it does two things: it **pushes the current PC** (the return address) onto the stack, then loads the PC with the function's address. When the function finishes with `RET`, the CPU **pops the return address** back into the PC — and execution resumes right where it left off.

```
        CALL readSensor    ; push return address, jump into readSensor
        MOV  A, R7         ; ← execution returns HERE afterwards

; ... elsewhere ...
readSensor:
        ; ... do the work, leave a result in R7 ...
        RET                ; pop return address, jump back to caller
```

Because the stack is LIFO, this works even when functions call functions call functions: each `CALL` pushes a return address, each `RET` pops the most recent one, and the chain unwinds perfectly in reverse order.

You can also use the stack yourself to *temporarily save* a value with `PUSH` and get it back later with `POP` — handy when you need a register for something else but must not lose what was in it:

```
        PUSH ACC           ; save the accumulator on the stack
        ; ... freely use A for something else ...
        POP  ACC           ; restore the original accumulator value
```

This same stack machinery is also what makes **interrupts** work: when a peripheral needs urgent attention, the CPU automatically pushes the PC, runs an interrupt routine, and pops the PC to resume — the program never even notices it was interrupted.

Part 7 — Putting it together

Let's connect every concept with one small, complete idea: a program that counts how many bytes in a small list are zero. It exercises the accumulator, a register, the flags, a conditional jump, and a loop.

```
        MOV  R0, #list     ; R0 points at the start of the data
        MOV  R2, #8        ; R2 = how many items to check (the loop counter)
        MOV  R7, #0        ; R7 = our running count of zeros (starts at 0)

next:   MOV  A, @R0        ; load the byte R0 points at into the accumulator
        JNZ  notzero       ; if A is NOT zero, skip the increment
        INC  R7            ; A was zero → count it
notzero:
        INC  R0            ; advance the pointer to the next byte
        DJNZ R2, next      ; R2 = R2 - 1; if not zero yet, loop again
        ; when we fall through here, R7 holds the number of zeros
```

Trace it in your head using what you now know:

- **R0** walks through memory; **R2** counts down the loop; **R7** accumulates the answer.
- `MOV A, @R0` fetches a byte into the **accumulator** — the scratchpad where the test happens.
- `JNZ` is a **conditional jump** that asks "is A zero?" and decides whether to run `INC R7`.

- `DJNZ` combines a decrement and a **conditional jump** to build the loop, changing the **program counter** to go back to `next` until the counter hits zero.
- Behind the scenes, every `MOV` and `INC` updates the CPU's state, and the whole thing runs inside the endless fetch–decode–execute cycle.

That is a real computer, doing real work, with nothing hidden. Everything you use at higher levels — Python lists, Arduino loops, whole operating systems — is built, ultimately, out of exactly these moves: load a register, test a flag, jump on a condition, push and pop the stack.

Glossary

Accumulator (A/ACC) — the CPU's main scratchpad register where most arithmetic and logic results appear.

Assembler — a tool that translates assembly-language mnemonics into machine-code bytes.

Assembly language — a human-readable, near one-to-one text form of machine code.

Carry flag (CY) — a status bit set when an unsigned calculation overflows past the byte's range.

Compiler — a tool that translates a high-level language (like C) into a lower-level one (assembly or machine code).

Conditional jump — an instruction that changes the program counter only if a condition (a flag, or A being zero) is true.

Flag — a single bit in the status register recording a fact about the CPU or the last operation.

Flash memory — non-volatile program memory that keeps its contents without power.

Linker — a tool that combines separately compiled object files and libraries into one final program, assigning final memory addresses.

Machine code — the raw bytes the CPU actually executes.

Microcontroller (MCU) — a complete small computer (CPU + memory + peripherals) on a single chip, designed to run one embedded program.

Overflow flag (OV) — like the carry flag, but for *signed* arithmetic.

Program counter (PC) — the register holding the address of the next instruction to execute.

Register — a tiny, very fast storage location built into the CPU.

Stack — a last-in-first-out region of memory used mainly to store return addresses for function calls.

Stack pointer (SP) — the register holding the address of the top of the stack.

Status register (PSW) — the register holding the CPU's flags.

Zero flag — the concept "was the result zero?"; note the classic 8051 has no zero-flag *bit* and instead tests the accumulator directly with `JZ` / `JNZ`.

Where to go next

This introduction gives you the mental model; the real knowledge comes from doing it yourself. A few concrete next steps:

- **Try the toolchain.** Install **SDCC** (the Small Device C Compiler) and a flashing tool such as `chprog` or the Arduino IDE with CH55x support. Compile a simple blink program and flash it to a CH552 board over USB. Deliberately watch the `.c` → compile → link → `.hex` → flash pipeline in action; it is exactly what we described in Part 2.
- **Read some assembly.** Ask SDCC to keep the generated assembly for a small C function (for example with the option that preserves an `.asm` output). Match each line of C to the jumps, register moves, and flags you now recognise. This is the fastest way to see what a compiler really does for you.
- **Play with the flags.** Write a small program that adds two numbers and then, depending on the carry flag, turns an LED on or off. This makes the abstract idea of a "flag" tangible.
- **Explore the peripherals.** The concepts in this document (registers, flags, reading and writing) extend directly to the special function registers (SFRs) that control GPIO, timers, PWM, and USB. Controlling hardware on a microcontroller is, ultimately, nothing more than reading and writing the right special registers.
- **Build something small.** Blink an LED, read a button, or send some text over USB to your PC. Each small project turns theory into something concrete and builds confidence.

Take your time, don't be afraid to break things (a fifty-cent microcontroller is forgiving), and always consult the official datasheet when the details matter.

Sources and further reading

- WCH, *CH552/CH551 8-bit Enhanced USB MCU Datasheet* — https://akizukidenshi.com/goodsaffix/CH552.pdf
- WCH, *CH552/CH551 Manual* — https://cdn.hackaday.io/files/1696717259204064/CH552%20Datasheet_C111367.zh-CN.en.pdf
- *Introduction to the CH552G Microcontroller* — https://altbier.us/ch552g/Intro_to_the_CH552G_Microcontroller.pdf
- CNX Software, *Cocket Nova CH552 development board* — https://www.cnx-software.com/2024/10/17/6-cocket-nova-ch552-development-board-features-ch552g-8-bit-mcu-with-an-enhanced-8051-core/

- SDCC C examples for the CH552 — https://github.com/Cesarbautista10/CH55x_SDCC_Examples